CSC311H5 Introduction to Machine Learning

Lecture 11

University of Toronto Mississauga

Unsupervised Learning

In the last two lectures, we will explore approaches for **unsupervised Learning**

No labeled examples; instead, looking for *interesting patterns* in the data

But what are *interesting patterns*?

Unsupervised Learning

In the last two lectures, we will explore approaches for **unsupervised Learning**

No labeled examples; instead, looking for *interesting patterns* in the data

But what are *interesting patterns*?

- Do data form *clusters*: i.e., are there groups of data points that are more similar to each other than to other data points?
- Can data be expressed in a lower-dimensional space?: i.e., do data points tend to fall in a *subspace* or *manifold* of the space that they inhabit?

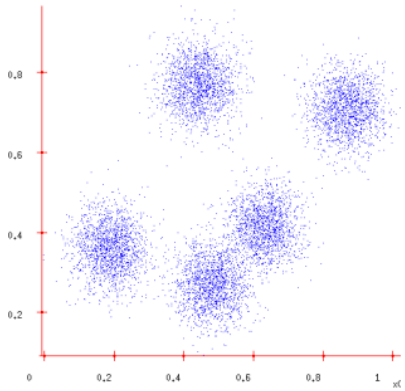
Today's Lecture on Clustering

First, introduce **k-means** a simple algorithm for **clustering**, i.e. grouping data points into clusters

Then, we will reformulate clustering as a latent variable model, apply the **Expectation-Maximization** algorithm

Clustering

Sometimes the data form clusters, where samples within a cluster are similar to each other, and samples in different clusters are dissimilar:

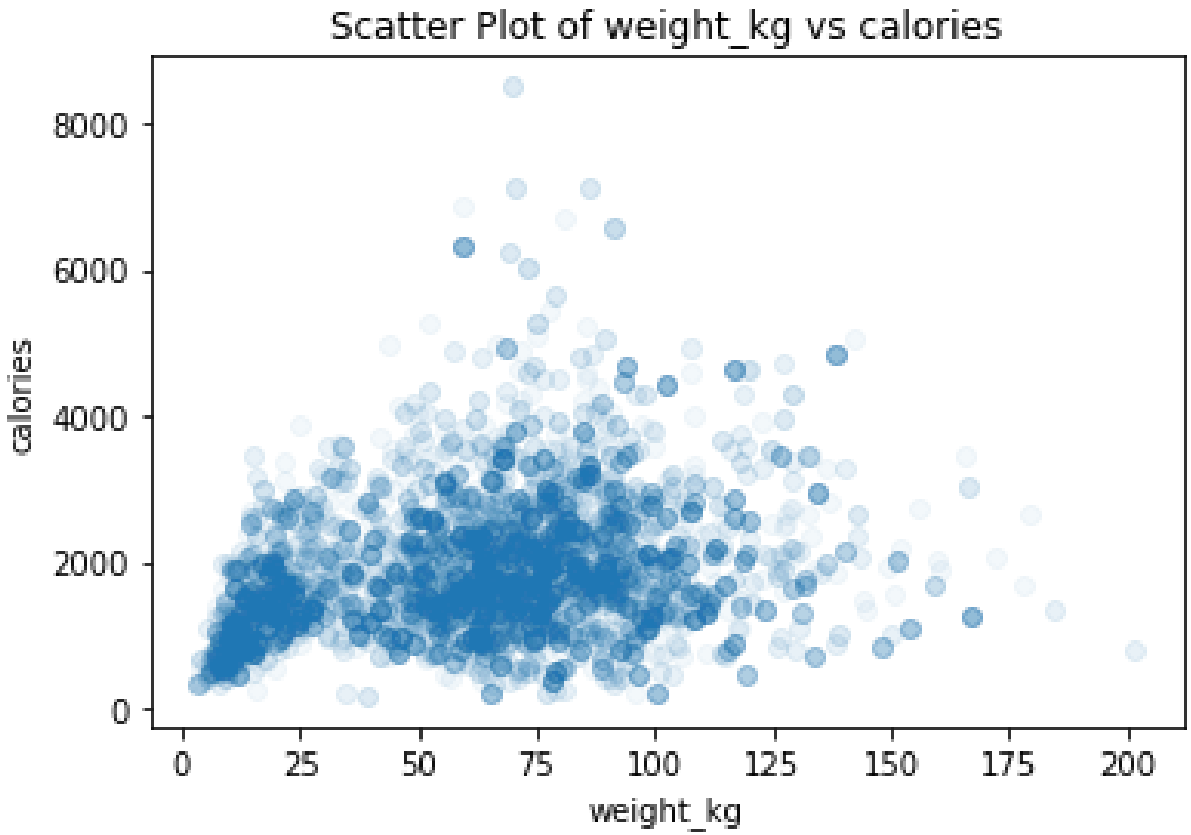


Such a distribution is **multimodal**, since it has multiple **modes**, or regions of high probability mass.

Grouping data points into clusters, **with no observed labels**, is called **clustering**. It is an unsupervised learning technique.

Example: clustering machine learning papers based on topic (deep learning, Bayesian models, etc.) *But topics are never observed (unsupervised)!*

Concrete Example from NHANES



What are the two axes? What are the "modes"? What clusters do you see?

Section 1

K-Means Clustering

k-means intuition

K-means assumes there are K clusters, and each point is close to its cluster **center**, or **mean** (the mean of points in the cluster).

- If we knew the cluster assignment, we could easily compute the centers.
- If we knew the centers, we could easily compute cluster assignment.

This is a chicken and egg problem!

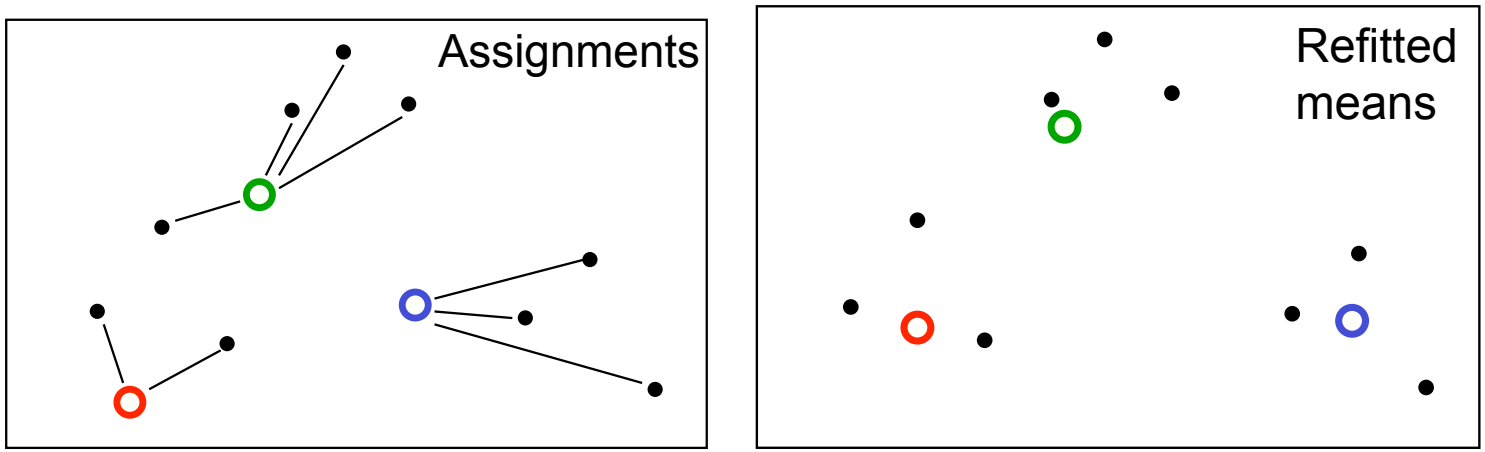
Very simple (and useful) heuristic - start randomly and alternate between the two!

K-means algorithm sketch

Initialization: randomly initialize cluster centers

The algorithm iteratively alternates between two steps:

- **Assignment step**: Assign each data point to the closest cluster
- **Refitting step**: Move each cluster center to the center of gravity of the data assigned to it



Exercise

Perform one iteration of k-means clustering on the data set in the handout. Initialize the cluster centers to $(0, 0)$ and $(5, 4)$.

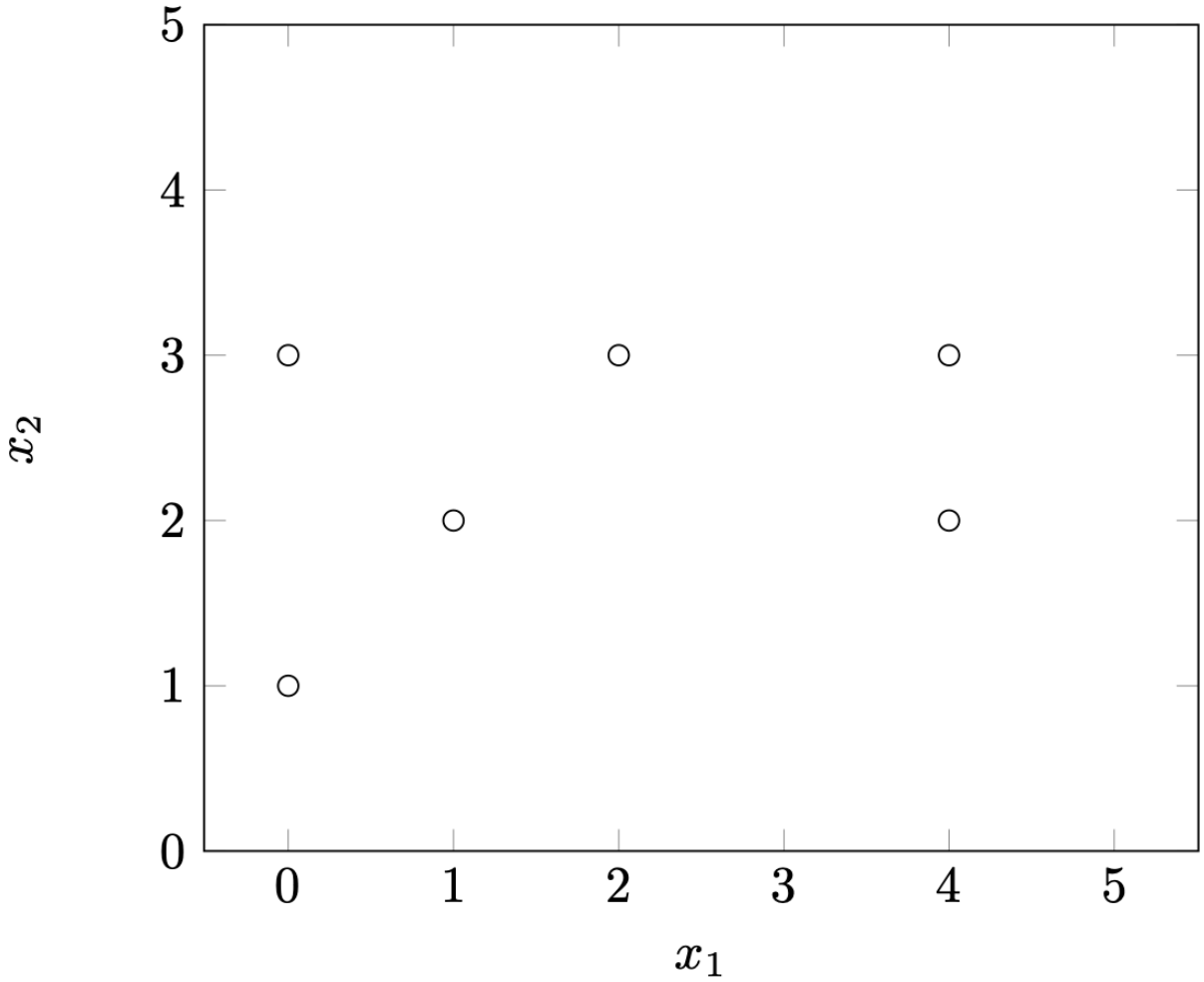
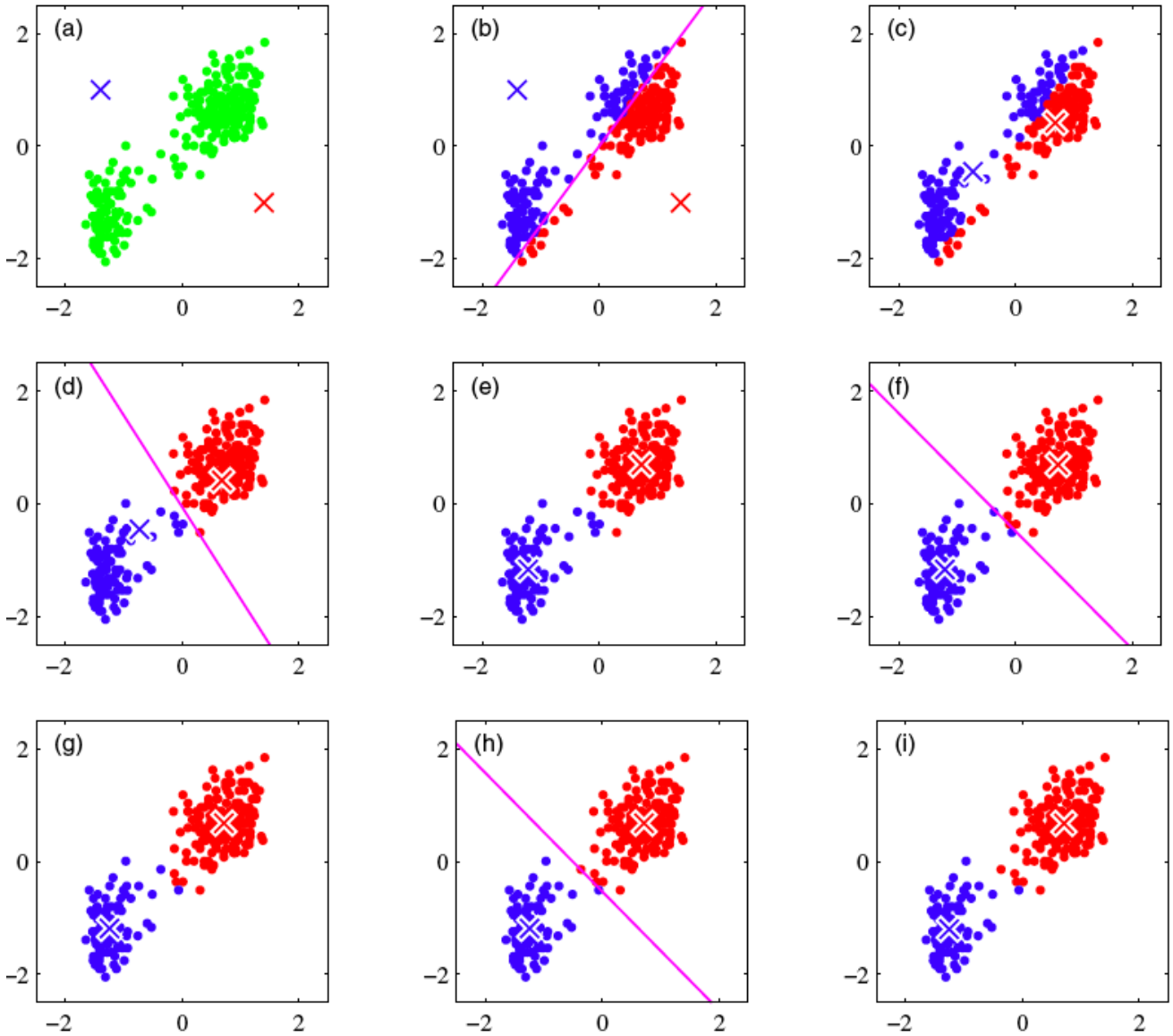


Figure from Bishop



What is actually being optimized?

k-means Objective: Find cluster centers $\mathbf{m}_k$ and assignments $\mathbf{r}^{(n)}$ to minimize the sum of squared distances of data points $\{\mathbf{x}^{(n)}\}$ to their assigned cluster centers:

$$\min_{\{\mathbf{m}_k\},\{\mathbf{r}^{(n)}\}} \sum_{n=1}^{N}\sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2$$

where $\mathbf{r}^{(n)}$ is a one-hot vector with $r_k^{(n)} \in \{0, 1\}$, and $r_k^{(n)} = 1$ means that $\mathbf{x}^{(n)}$ is assigned to cluster k (with center $\mathbf{m}_k$).

That is, if $\mathbf{x}^{(n)}$ is assigned to cluster $\hat{k}$,

$$\mathbf{r}^{(n)} = \underbrace{[0, 0, ..., 1, ..., 0]^\top}_{\text{Only } \hat{k}\text{-th entry is 1}}$$

K-Means Objective

$$\min_{\{\mathbf{m}_k\},\{\mathbf{r}^{(n)}\}} \sum_{n=1}^{N}\sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2$$

- Finding the exact optimum can be shown to be NP-hard.
- K-means can be seen as block coordinate descent on this objective
  - **Assignment step**: minimize with respect to $\{r_k^{(n)}\}$
  - **Refitting step**: minimize with respect to $\{\mathbf{m}_k\}$

How to optimize?: Alternating Minimization

$$\text{Optimization problem:} \quad \min_{\{\mathbf{m}_k\},\{\mathbf{r}^{(n)}\}} \sum_{n=1}^{N}\sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2$$

If we fix the centers $\{\mathbf{m}_k\}$ then we can easily find the optimal assignments $\{\mathbf{r}^{(n)}\}$ for each sample n,

$$\min_{\mathbf{r}^{(n)}} \sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2$$

Assignment Step: Assign each point to the cluster with the nearest center

$$r_k^{(n)} = \left\{ \begin{array}{ll} 1 & \text{if } k = \arg\min_j \|\mathbf{x}^{(n)} - \mathbf{m}_j\|^2 \\ 0 & \text{otherwise} \end{array} \right\}$$

Alternating Minimization

Likewise, if we fix the assignments $\{\mathbf{r}^{(n)}\}$ then can easily find optimal centers $\{\mathbf{m}_k\}$

Refitting Step: Minimize the objective by finding its critcial point.

$$\begin{aligned}\mathbf{0} =& \nabla_{\mathbf{m}_\ell} \left(\sum_{n=1}^{N} \sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2 \right) \\ =& 2 \sum_{n=1}^{N} r_\ell^{(n)} (\mathbf{m}_\ell - \mathbf{x}^{(n)}) \quad \implies \mathbf{m}_\ell = \frac{\sum_n r_\ell^{(n)} \mathbf{x}^{(n)}}{\sum_n r_\ell^{(n)}}\end{aligned}$$

Alternating Minimization

Likewise, if we fix the assignments $\{\mathbf{r}^{(n)}\}$ then can easily find optimal centers $\{\mathbf{m}_k\}$

Refitting Step: Minimize the objective by finding its critcial point.

$$\begin{aligned}\mathbf{0} =& \nabla_{\mathbf{m}_\ell} \left(\sum_{n=1}^{N} \sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2 \right) \\ =& 2 \sum_{n=1}^{N} r_\ell^{(n)} (\mathbf{m}_\ell - \mathbf{x}^{(n)}) \quad \implies \mathbf{m}_\ell = \frac{\sum_n r_\ell^{(n)} \mathbf{x}^{(n)}}{\sum_n r_\ell^{(n)}}\end{aligned}$$

K-Means simply alternates bewteen minimizing with respect to **assignments** and **centers**.

This is an instance of **alternating minimization**, or **block coordinate descent**.

The K-means Algorithm

Initialization: Set K cluster means $\mathbf{m}_1, \ldots, \mathbf{m}_K$ to random values

Repeat until convergence (until assignments do not change):

- **Assignment**: Optimize the objective with respect to $\{\mathbf{r}\}$: Each data point $\mathbf{x}^{(n)}$ assigned to nearest center

$$\hat{k}^{(n)} = \arg\min_k ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2$$

  and set the **responsibilities** accordingly:

$$r_k^{(n)} = \mathbb{I}[\hat{k}^{(n)} = k] \quad \text{for} \quad k = 1, .., K$$

- **Refitting:** Optimize the objective with respect to $\{\mathbf{m}\}$: Each center is set to mean of the data assigned to it

$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}$$

Why K-means Converges

$$\text{K-means Cost: } \sum_{n=1}^{N} \sum_{k=1}^{K} r_k^{(n)} ||\mathbf{m}_k - \mathbf{x}^{(n)}||^2$$

K-means algorithm reduces the cost at each iteration.

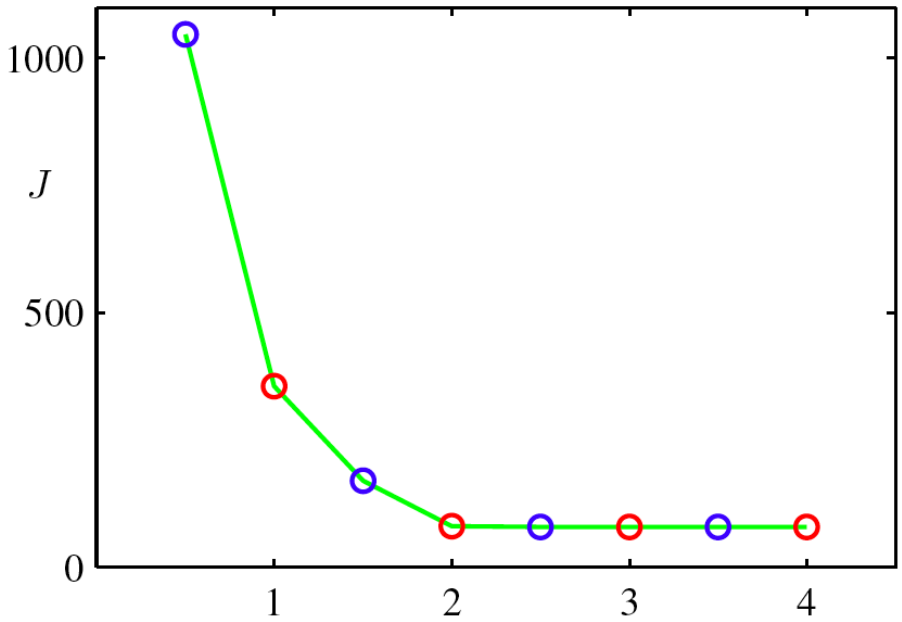
- Whenever an assignment is changed, the sum squared distances of data points from their assigned cluster centers is reduced.
- Whenever a cluster center is moved, the total cost is reduced.

Test for convergence: If the assignments do not change in the assignment step, we have converged (to at least a local minimum).

- This will always happen after a finite number of iterations, since the number of possible cluster assignments is finite

K-means cost across iterations

K-means cost function after each assignment step (blue) and refitting step (red).

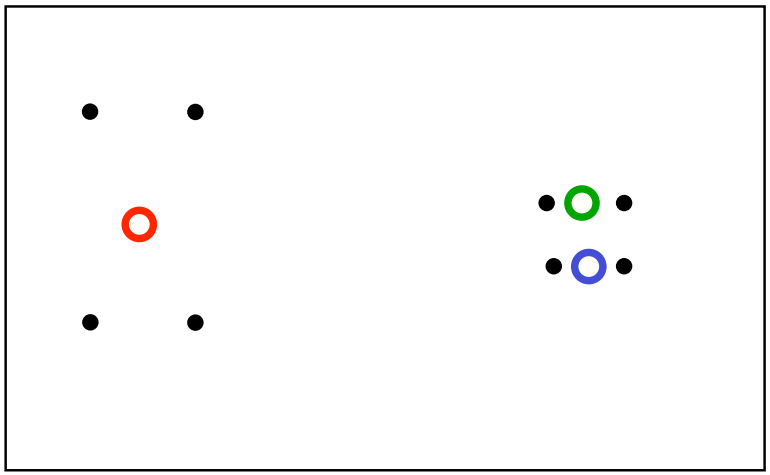


The algorithm has converged after the third refitting step.

Local Minima

- The objective is non-convex (so coordinate descent is not guaranteed to converge to the global minimum)
- There is nothing to prevent k-means getting stuck at local minima.
- We could try many random starting points

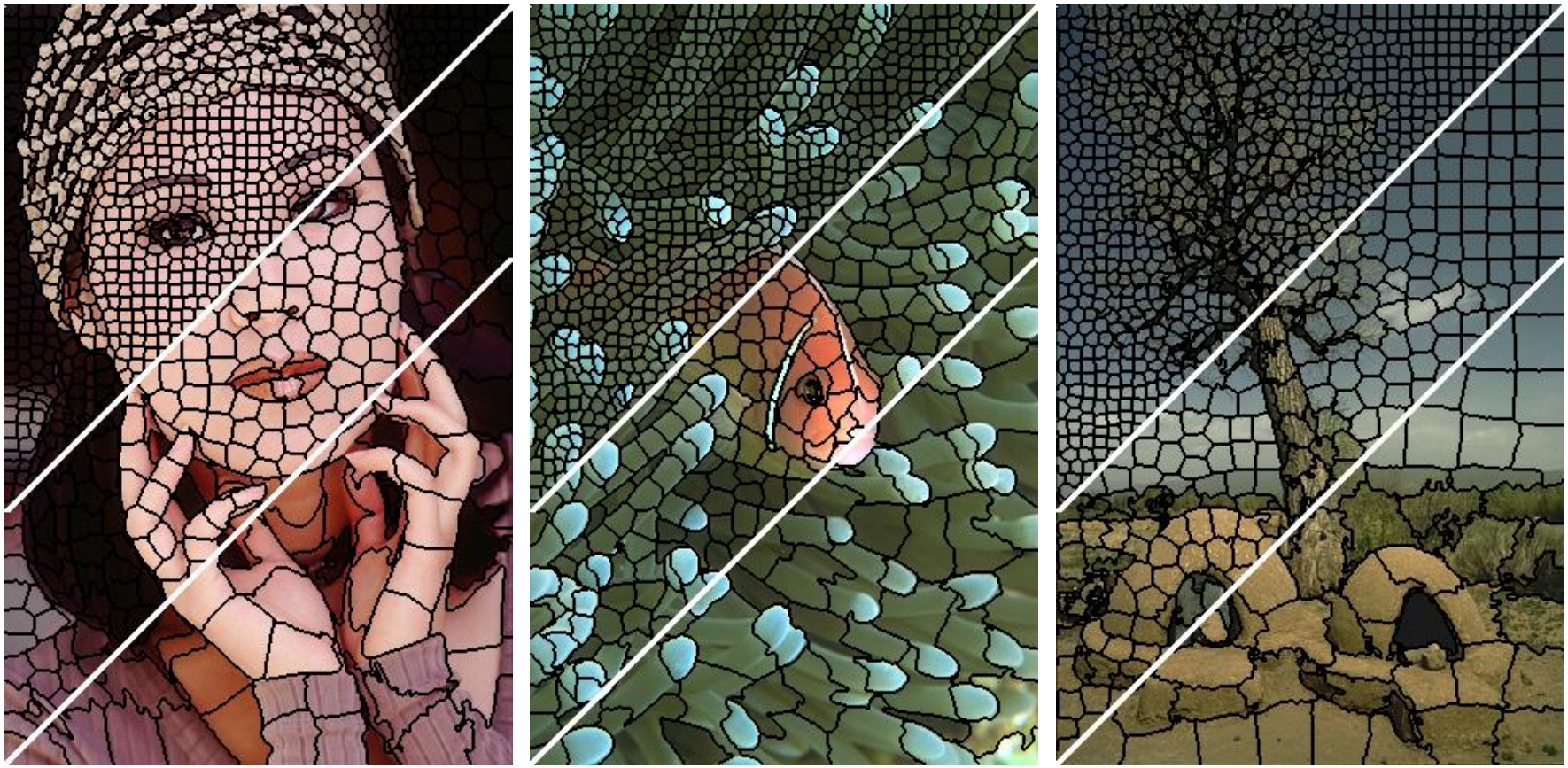
A bad local optimum



Example: K-means for Vector Quantization

- Given image, construct "dataset" of pixels represented by their RGB pixel intensities
- Run k-means, replace each pixel by its cluster center

Example: K-means for Image Segmentation



- Given image, construct "dataset" of pixels, represented by their RGB pixel intensities and grid locations
- Run k-means (with some modifications) to get *superpixels*

Soft K-means

Instead of making hard assignments of data points to clusters, we can make **soft assignments**. One cluster may have a responsibility of 0.7 for a data point and another may have a responsibility of 0.3.

- Allows a cluster to use more information about the data in the refitting step.
- How do we decide on the soft assignments?
- We already saw this in multi-class classification: one-hot encoding vs softmax assignments

Soft K-means Algorithm

Initialization: Set K means $\{\mathbf{m}_k\}$ to random values

Repeat until convergence (measured by how much the cost changes):

- **Assignment**: Each data point n given soft "degree of assignment" to each cluster mean k, based on responsibilities

$$r_k^{(n)} = \frac{\exp[-\beta \|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2]}{\sum_{j=1}^{K} \exp[-\beta \|\mathbf{m}_j - \mathbf{x}^{(n)}\|^2]}$$

$$\implies \mathbf{r}^{(n)} = \text{softmax}(-\beta \{\|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2\}_{k=1}^{K})$$

- **Refitting:** Model parameters, means, are adjusted to match sample means of data points they are responsible for:

$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}$$

Soft K-means Algorithm

Initialization: Set K means $\{\mathbf{m}_k\}$ to random values

Repeat until convergence (measured by how much the cost changes):

- **Assignment**: Each data point n given soft "degree of assignment" to each cluster mean k, based on responsibilities

$$r_k^{(n)} = \frac{\exp[-\beta \|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2]}{\sum_{j=1}^{K} \exp[-\beta \|\mathbf{m}_j - \mathbf{x}^{(n)}\|^2]}$$

$$\implies \mathbf{r}^{(n)} = \text{softmax}(-\beta \{\|\mathbf{m}_k - \mathbf{x}^{(n)}\|^2\}_{k=1}^{K})$$

- **Refitting:** Model parameters, means, are adjusted to match sample means of data points they are responsible for:

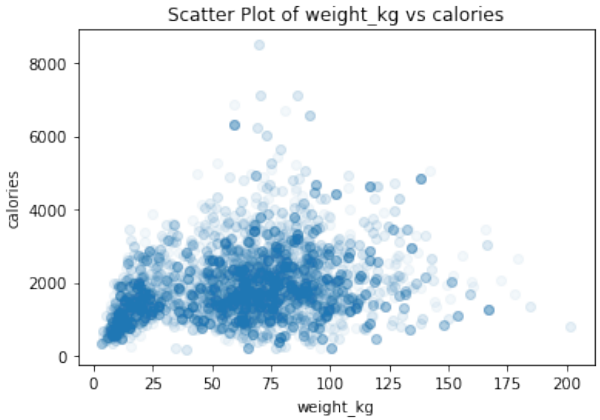
$$\mathbf{m}_k = \frac{\sum_n r_k^{(n)} \mathbf{x}^{(n)}}{\sum_n r_k^{(n)}}$$

Note: As $\beta \to \infty$, soft k-Means becomes k-Means! (Why?)

Questions about Soft K-means

Some remaining issues

- How to set β?
- Clusters with unequal weight and width?



These aren't straightforward to address with K-means. Instead, in the sequel, we'll reformulate clustering using a generative model.

Section 2

Gaussian Mixture Models

A Generative View on Clustering

Remainder of this lecture: formulating clustering as a probabilistic model

- specify assumptions about how the observations relate to latent variables
- use an algorithm called Expectation-Maximization to (approximately) maximize the likelihood of the observations

This lets us generalize clustering to non-spherical centers or to non-Gaussian observation models

This lecture is when probabilistic modeling starts to shine!

From Generative Models to Latent Variable Models

Recall generative (Bayes) classifiers: $p(\mathbf{x}, t) = p(\mathbf{x}|t)\, p(t)$ where we fit $p(t)$ and $p(\mathbf{x}|t)$ using labeled data.

Idea: our model for how the data is generated is that

- First we sample t from the distribution $p(t)$
- Then, given that t, we sample $\mathbf{x}$ from the distribution $p(\mathbf{x}|t)$

From Generative Models to Latent Variable Models

Recall generative (Bayes) classifiers: $p(\mathbf{x}, t) = p(\mathbf{x}|t)\, p(t)$ where we fit $p(t)$ and $p(\mathbf{x}|t)$ using labeled data.

Idea: our model for how the data is generated is that

- First we sample t from the distribution $p(t)$
- Then, given that t, we sample $\mathbf{x}$ from the distribution $p(\mathbf{x}|t)$

If t is never observed, we call it a **latent variable**, or **hidden variable**, and generally denote it with z (instead of t):

$$p(\mathbf{x}, z) = p(\mathbf{x}|z)\, p(z)$$

The things we *can* observe (i.e., $\mathbf{x}$) are called **observables**.

Latent Variable Models

By marginalizing out z, we get a density over the observables:

$$p(\mathbf{x}) = \sum_z p(\mathbf{x}, z) = \sum_z p(\mathbf{x}|z)\, p(z)$$

This is called a **latent variable model**.

If $p(z)$ is a categorical distribution, this is a **mixture model**, and different values of z correspond to different **components**.

Gaussian Mixture Model (GMM)

A GMM represents a distribution as a *mixture of Gaussians*

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

with π_k the **mixing coefficients**, where:

$$\sum_{k=1}^{K} \pi_k = 1 \quad \text{and} \quad \pi_k \geq 0 \quad \forall k$$

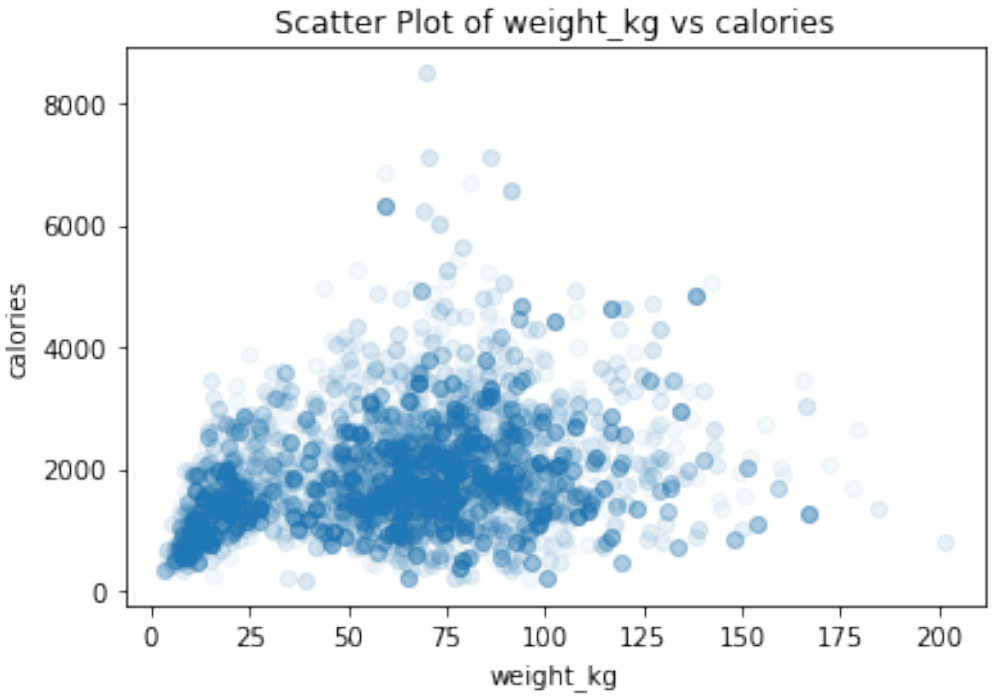
This defines a density over $\mathbf{x}$, so we can fit the parameters using maximum likelihood. We try to match the data density of $\mathbf{x}$ as closely as possible.

- This is a hard optimization problem (and the focus of this lecture).

GMM as Universal Approximators

GMMs are **universal approximators of densities** (analogously to our universality result for MLPs). Even diagonal GMMs are universal approximators.

As a concrete example, we can fit a GMM to approximate this density, as we could any other:



GMM as a Generative Process

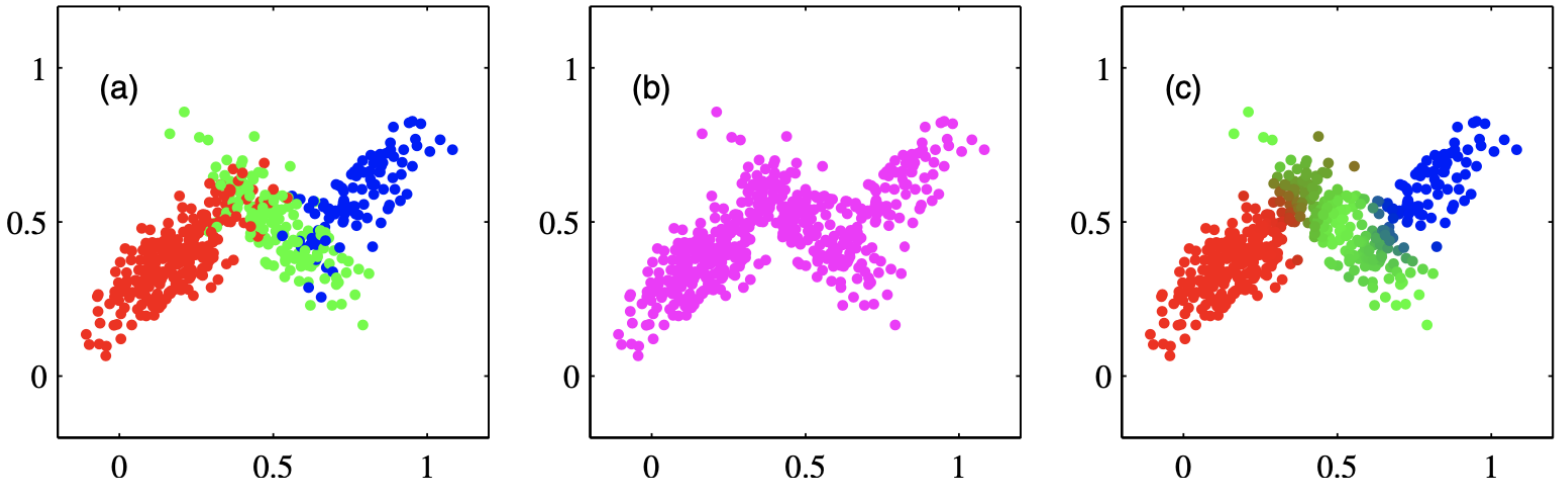
We can also write the model as a **generative process**:

For $i = 1, \ldots, N$:

- $z^{(i)} \sim \text{Categorical}(\boldsymbol{\pi})$
  - choose a "cluster" (i.e., a Gaussian) from which to sample $\mathbf{x}^{(i)}$
- $\mathbf{x}^{(i)} \sim \mathcal{N}(\boldsymbol{\mu}_{z^{(i)}}, \boldsymbol{\Sigma}_{z^{(i)}})$
  - sample the observable features $\mathbf{x}^{(i)}$ from the chosen Gaussian

The Generative Model: Example

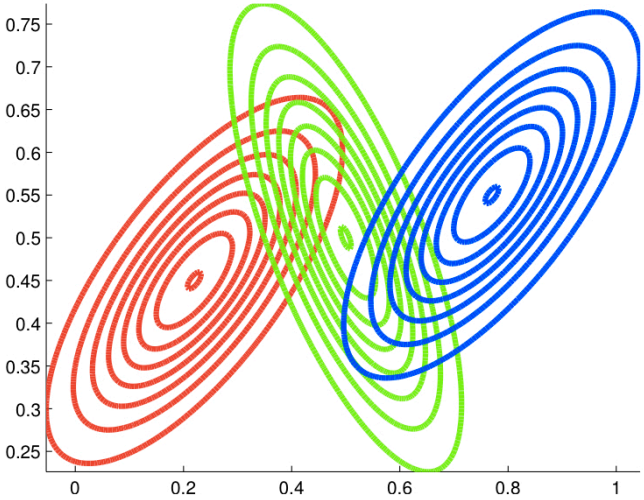
Here are 500 points drawn from a mixture of 3 (2D) Gaussians.

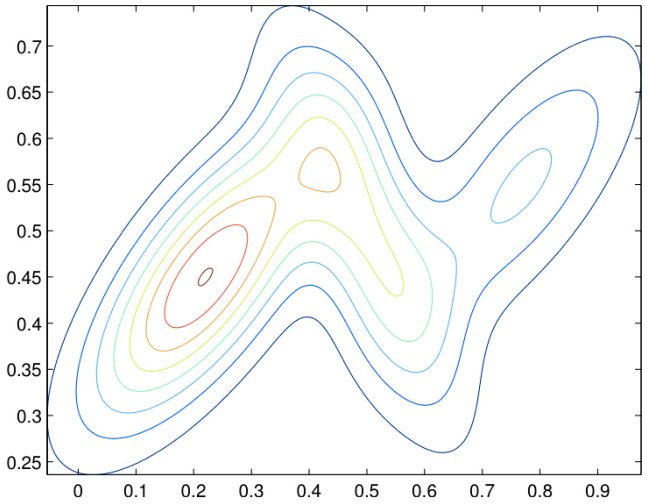


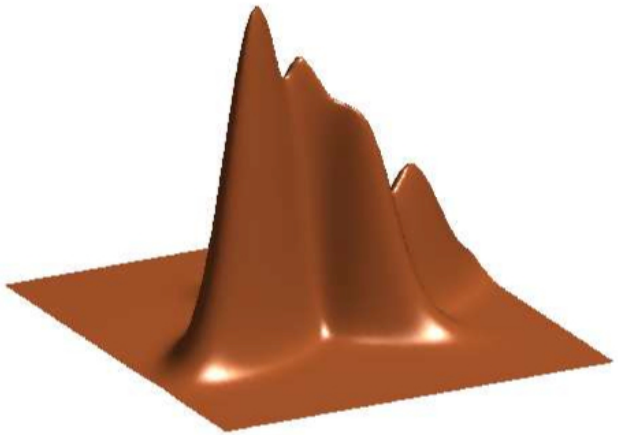
a) Samples from $p(\mathbf{x}|z)$ b) Samples from the marginal $p(\mathbf{x})$ c) Responsibilities $p(z|\mathbf{x})$

Mixture of Gaussians – 2D Gaussians

Visualizing Densities

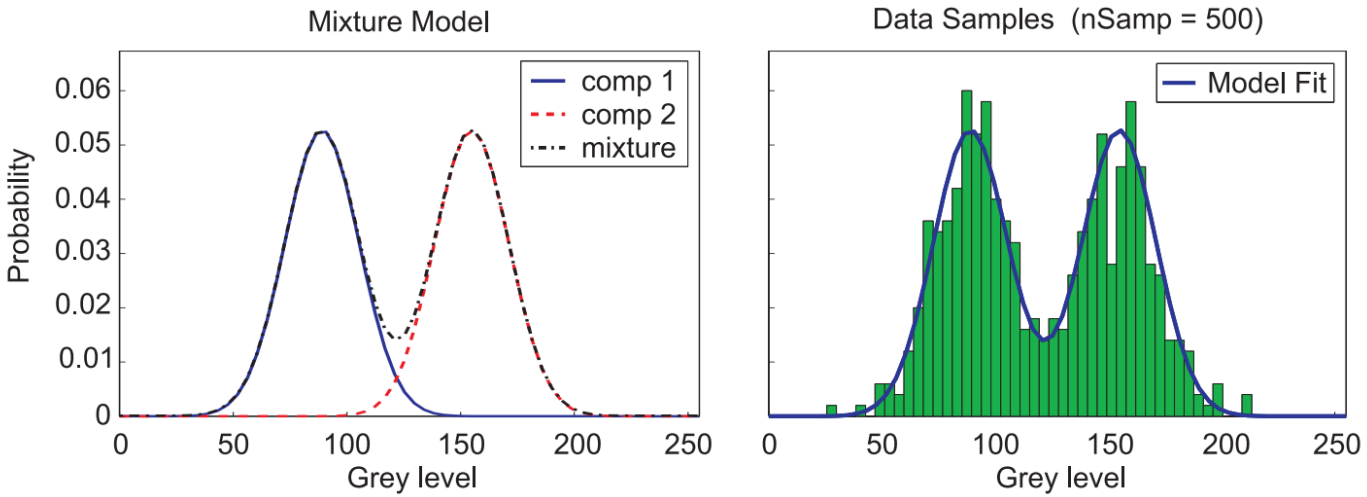






Mixture of Gaussians – 1D Gaussians

Example of the density of a GMM with $K = 2$ Gaussians:



Fitting a Latent Variable Model?

How should we choose the parameters $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$?

Fitting a Latent Variable Model?

How should we choose the parameters $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$?

Maximum likelihood principle: choose parameters θ to maximize likelihood of observed data

- We don't observe the cluster assignments z, we only see the data $\mathbf{x}$
- Given data $\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$, choose parameters θ to maximize:

$$\log p(\mathcal{D}; \theta) = \sum_{n=1}^{N} \log p(\mathbf{x}^{(n)}; \theta)$$

- We can express $p(\mathbf{x}; \theta)$ by marginalizing out z:

$$p(\mathbf{x}; \theta) = \sum_{k=1}^{K} P(z = k, \mathbf{x}; \theta) = \sum_{k=1}^{K} P(z = k; \theta) p(\mathbf{x} | z = k; \theta)$$

Section 3

Expectation Maximization (E-M)

Fitting GMMs: Maximum Likelihood

Some shorthand notation: let $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$ denote the full set of model parameters. Let $\mathbf{X} = \{\mathbf{x}^{(i)}\}_{i=1}^N$ and $\mathbf{z} = \{z^{(i)}\}_{i=1}^N$.

GMM Maximum Likelihood objective:

$$\log p(\mathbf{X}; \theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$$

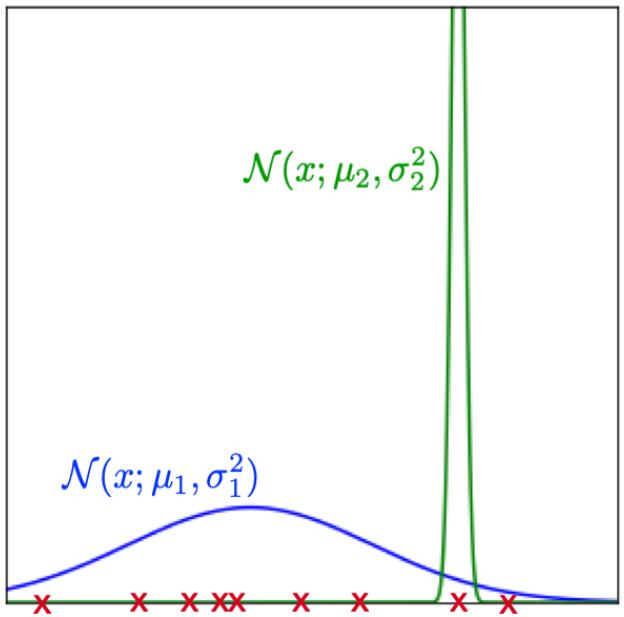
- In general, no closed-form solution
- Not **identifiable**: solution is invariant to permutations
- Challenges in optimizing this using gradient descent?
  - Non-convex (due to permutation symmetry)
  - Need to enforce non-negativity constraint on π_k
  - Need the covariance matrices $\boldsymbol{\Sigma}_k$ to be positive semi-definite
  - Derivatives with respect to $\boldsymbol{\Sigma}_k$ are expensive/complicated.

We need a different approach!

Fitting GMMs: Degenerate Solutions

Warning: We don't want the global maximum. We can achieve arbitrarily high training likelihood by placing a small-variance Gaussian component on a training example.

This is known as a **singularity**.



Latent Variable Models: Inference

Goal: maximize $\log p(\mathbf{X}; \theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$

If we knew the parameters $\theta = \{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}$, we could infer which component a data point $\mathbf{x}$ probably belongs to by inferring its latent variable z.

This is just posterior inference, which we do using Bayes' Rule:

$$P(z = k|\mathbf{x}) = \frac{P(z = k)\, p(\mathbf{x}|z = k)}{\sum_{\ell} P(z = \ell)\, p(\mathbf{x}|z = \ell)}$$

This is just like GDA at test time!!

Latent Variable Models: Learning

Goal: maximize $\log p(\mathbf{X}; \theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$

If we somehow knew the latent variables for every data point, we could simply maximize the joint log-likelihood.

$$\begin{aligned} \log p(\mathbf{X}, \mathbf{Z}; \theta) &= \sum_{i=1}^{N} \log p(\mathbf{x}^{(i)}, z^{(i)}; \theta) \\ &= \sum_{i=1}^{N} \log p(z^{(i)}) + \log p(\mathbf{x}^{(i)} | z^{(i)}). \end{aligned}$$

This is just like GDA at training time!!

Latent Variable Models: Learning (2)

Goal: maximize $\log p(\mathbf{X}; \theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$

Here's our formulas from GDA, written in a suggestive notation:

$$\begin{aligned} \pi_k &= \frac{1}{N} \sum_{i=1}^{N} r_k^{(i)}, \quad \boldsymbol{\mu}_k = \frac{\sum_{i=1}^{N} r_k^{(i)} \cdot \mathbf{x}^{(i)}}{\sum_{i=1}^{N} r_k^{(i)}} \\ \boldsymbol{\Sigma}_k &= \frac{1}{\sum_{i=1}^{N} r_k^{(i)}} \sum_{i=1}^{N} r_k^{(i)} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top \\ r_k^{(i)} &= \mathbb{I}[z^{(i)} = k] \end{aligned}$$

Here we assume we know $r_k^{(1)}$, as in GDA.

Latent Variable Models

Goal: maximize $\log p(\mathbf{X}; \theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right)$

We have a chicken-and-egg problem, just like with K-Means!

- Given θ, inferring $z^{(i)}$ (with the Bayes rule) is easy.
- Given the $z^{(i)}$, learning θ (with maximum likelihood) is easy.

Doing both simultaneously is hard!

- We don't observe the $z^{(i)}$s
- The log above is outside the sum, so things don't simplify.

Can you guess the algorithm?

Expectation-Maximization Intuition

We use the **Expectation-Maximization algorithm**, which alternates between two steps:

- **Expectation step (E-step)**: Compute the posterior probability over z given our current model - i.e. how much do we think each Gaussian generates each data point.

Expectation-Maximization Intuition

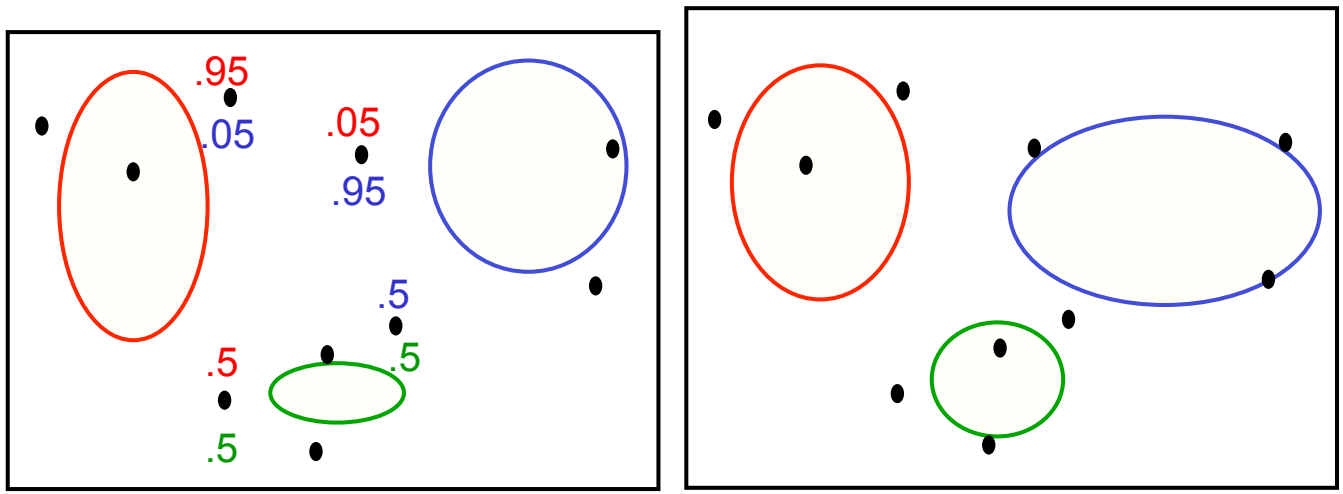
We use the **Expectation-Maximization algorithm**, which alternates between two steps:

- **Expectation step (E-step)**: Compute the posterior probability over z given our current model - i.e. how much do we think each Gaussian generates each data point.
- **Maximization step (M-step)**: Assuming that the data really was generated this way, change the parameters of each Gaussian to maximize the probability that it would generate the data it is currently responsible for.

Expectation-Maximization Intuition

We use the **Expectation-Maximization algorithm**, which alternates between two steps:

- **Expectation step (E-step)**: Compute the posterior probability over z given our current model - i.e. how much do we think each Gaussian generates each data point.
- **Maximization step (M-step)**: Assuming that the data really was generated this way, change the parameters of each Gaussian to maximize the probability that it would generate the data it is currently responsible for.



Expectation Maximization for GMM Overview

E-step: Assign the **responsibility** $r_k^{(i)}$ of component k for data point i using the posterior probability:

$$r_k^{(i)} = P(z^{(i)} = k | \mathbf{x}^{(i)}; \theta)$$

M-step: Apply the maximum likelihood updates, where each component is fit with a weighted dataset. The weights are proportional to the responsibilities.

$$\begin{aligned}
\pi_k &= \frac{1}{N} \sum_{i=1}^{N} r_k^{(i)} \\
\boldsymbol{\mu}_k &= \frac{\sum_{i=1}^{N} r_k^{(i)} \cdot \mathbf{x}^{(i)}}{\sum_{i=1}^{N} r_k^{(i)}} \\
\boldsymbol{\Sigma}_k &= \frac{1}{\sum_{i=1}^{N} r_k^{(i)}} \sum_{i=1}^{N} r_k^{(i)} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top
\end{aligned}$$

Temperature GMM Exercise (E-Step)

Exercise Suppose we have the following measures for the temperature in Toronto this month:

$$x^{(1)} = 10, x^{(2)} = 12, x^{(3)} = 5, x^{(4)} = 4, x^{(5)} = 9$$

We wish to fit a mixture of 2 Gaussians to this data set.

Suppose we initialize $\mu_1 = 2$, $\mu_2 = 7$, $\sigma_1 = \sigma_2 = 1$, and $\pi_1 = \pi_2 = 0.5$. What is the value of the responsibility vector $\mathbf{r}^{(1)}$ for example 1? Recall that the PDF of the univariate Gaussian is

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Temperature GMM Exercise (E-Step)

Exercise Suppose we have the following measures for the temperature in Toronto this month:

$$x^{(1)} = 10, x^{(2)} = 12, x^{(3)} = 5, x^{(4)} = 4, x^{(5)} = 9$$

We wish to fit a mixture of 2 Gaussians to this data set.

Suppose we initialize $\mu_1 = 2$, $\mu_2 = 7$, $\sigma_1 = \sigma_2 = 1$, and $\pi_1 = \pi_2 = 0.5$. What is the value of the responsibility vector $\mathbf{r}^{(1)}$ for example 1? Recall that the PDF of the univariate Gaussian is

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

To get started, use the Bayes Rule:

$$\begin{aligned} r_k^{(1)} &= P(z^{(1)} = k | x^{(1)}; \theta) \\ &= \frac{p(x^{(1)}|z^{(1)} = 0)p(z^{(1)} = 0)}{p(x^{(1)}|z^{(1)} = 1)p(z^{(1)} = 1) + p(x^{(1)}|z^{(1)} = 2)p(z^{(1)} = 2)} \end{aligned}$$

Temperature GMM Exercise (E-Step)

This is the same as the inference question from the math prepare!

$$
\begin{aligned}
p(x^{(1)} = 10 | z^{(1)} = 1) p(z^{(1)} = 1) &= \mathcal{N}(10; \mu = 2, \sigma = 1)\pi_1 \\
&\approx 5.052271083536893e-15 \\
p(x^{(1)} = 10 | z^{(1)} = 2) p(z^{(1)} = 2) &= \mathcal{N}(10; \mu = 7, \sigma = 1)\pi_2 \\
&\approx 0.00443184841193800 75
\end{aligned}
$$

Thus, $r_1^{(1)} \approx 0$, and $r_2^{(1)} \approx 1$.

Temperature GMM Exercise (M-Step)

Exercise Suppose we have the following measures for the temperature in Toronto this month:

$$x^{(1)} = 10, x^{(2)} = 12, x^{(3)} = 5, x^{(4)} = 4, x^{(5)} = 9$$

We wish to fit a mixture of 2 Gaussians to this data set.

Suppose that we have

$$\mathbf{r}^{(1)} \approx \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{r}^{(2)} \approx \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{r}^{(3)} \approx \begin{bmatrix} 0.08 \\ 0.92 \end{bmatrix}, \quad \mathbf{r}^{(4)} \approx \begin{bmatrix} 0.92 \\ 0.08 \end{bmatrix}, \quad \mathbf{r}^{(5)} \approx \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Re-estimate μ_1, μ_2, σ_1, σ_2, π_1 and π_2.

Temperature GMM Exercise (M-Step)

$$\pi_1 = \frac{1}{5}\sum_{i=1}^{5} r_1^{(i)} = \dots$$

$$\pi_2 = \frac{1}{5}\sum_{i=1}^{5} r_2^{(i)} = 1 - \pi_1$$

$$\mu_1 = \frac{\sum_{i=1}^{5} r_1^{(i)} \cdot x^{(i)}}{\sum_{i=1}^{5} r_1^{(i)}} = \dots$$

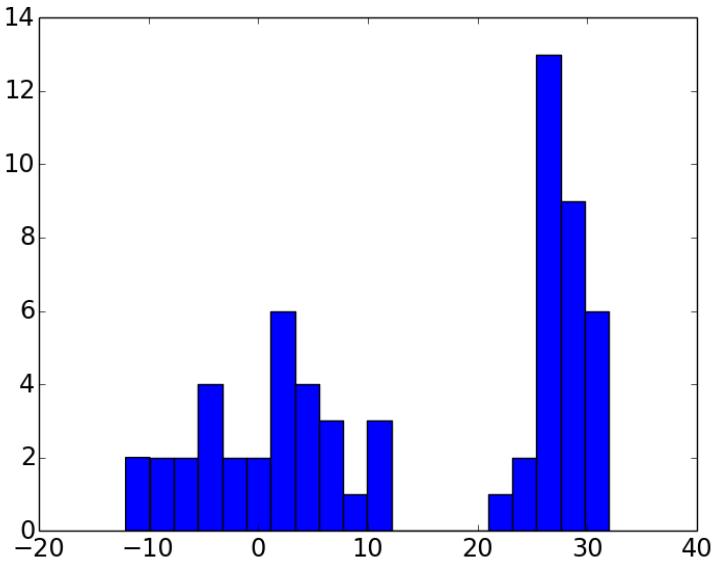
$$\mu_2 = \frac{\sum_{i=1}^{5} r_2^{(i)} \cdot x^{(i)}}{\sum_{i=1}^{5} r_2^{(i)}} = \dots$$

$$\sigma_1 = \frac{1}{\sum_{i=1}^{5} r_1^{(i)}} \sum_{i=1}^{5} r_1^{(i)} (x^{(i)} - \mu_1)(x^{(i)} - \mu_1) = \dots$$

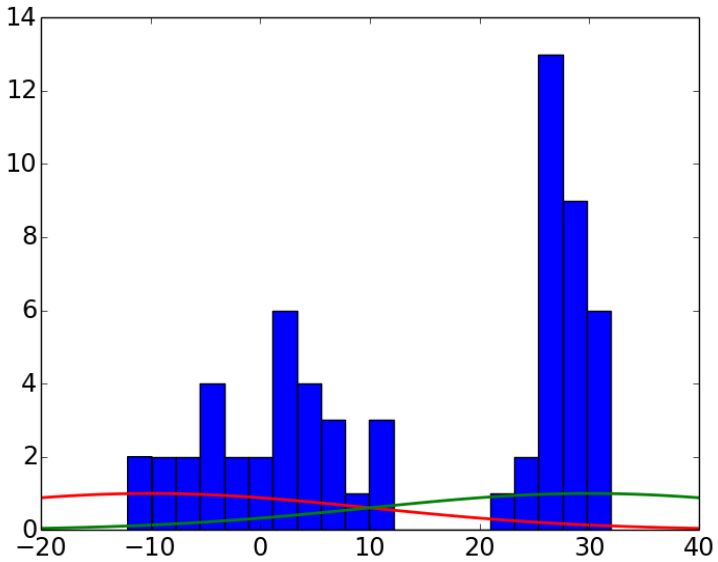
$$\sigma_2 = \frac{1}{\sum_{i=1}^{5} r_2^{(i)}} \sum_{i=1}^{5} r_2^{(i)} (x^{(i)} - \mu_2)(x^{(i)} - \mu_2) = \dots$$

Temperature Example

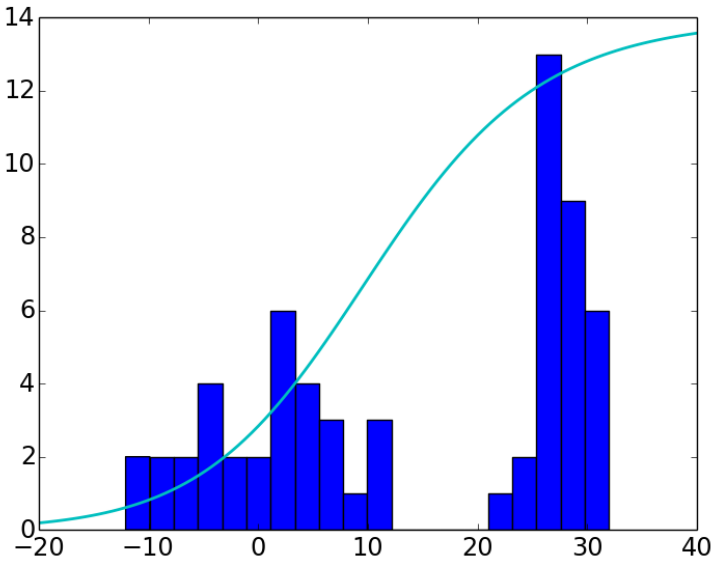
Let's demonstrate how GMM would work with the same problem if we had some more data:

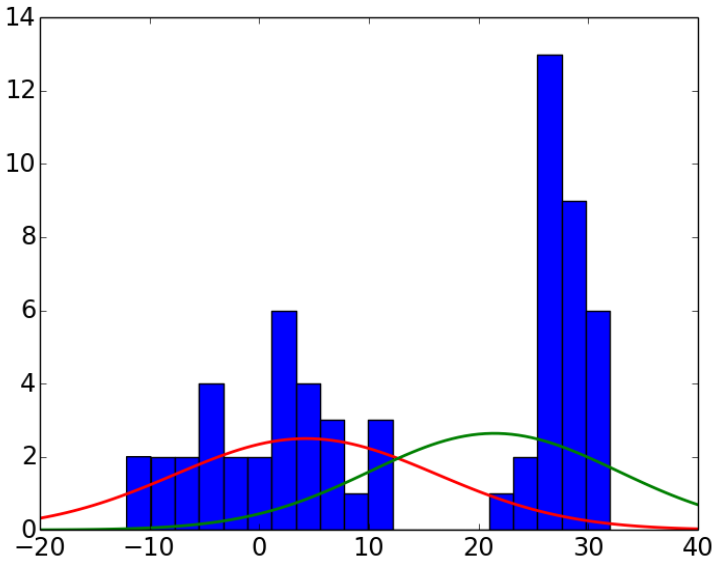


Temperature Example: Initialization

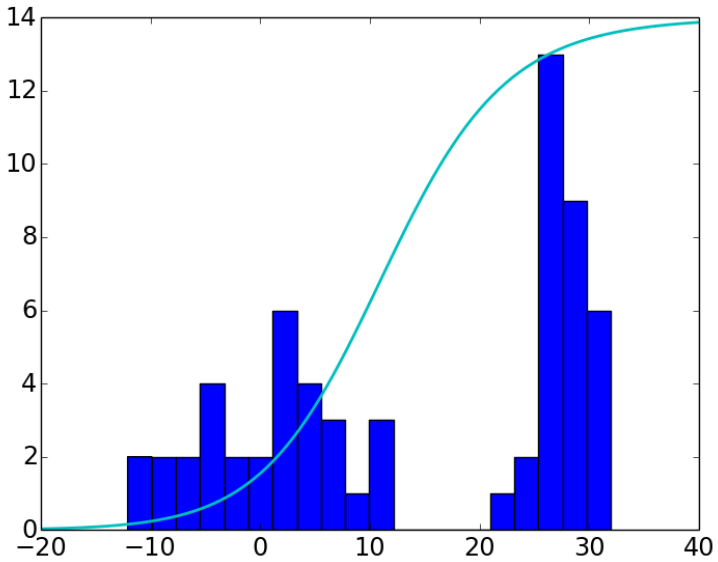


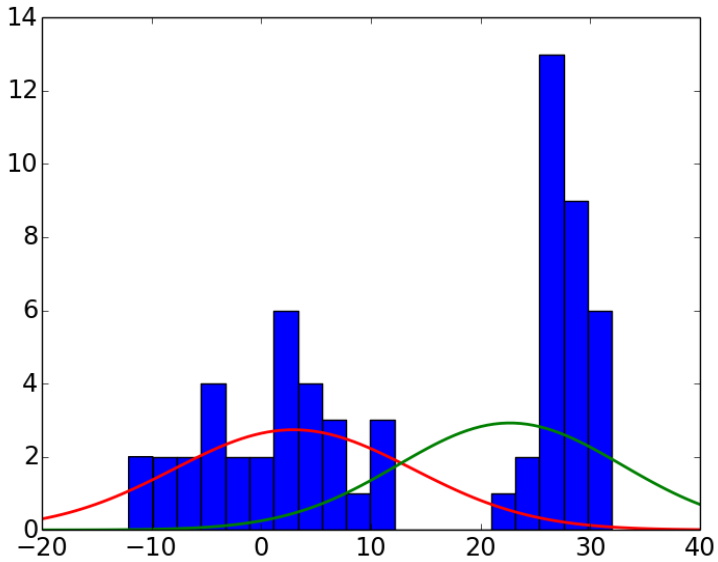
Temperature Example: Step 1



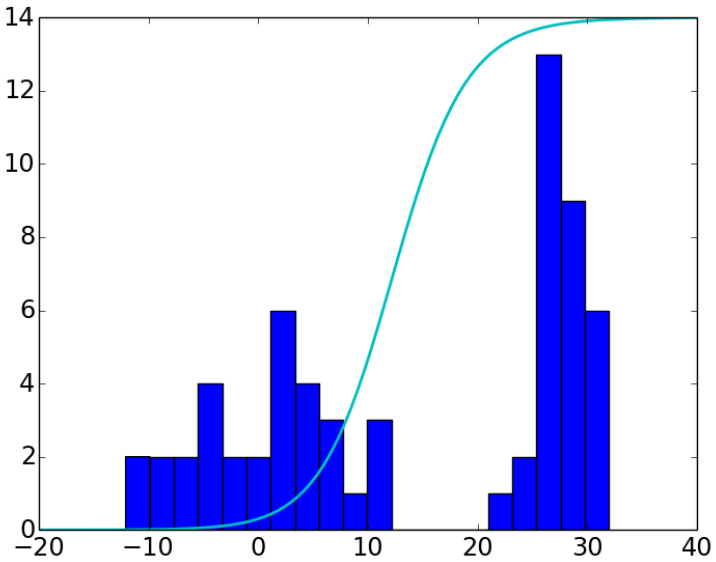


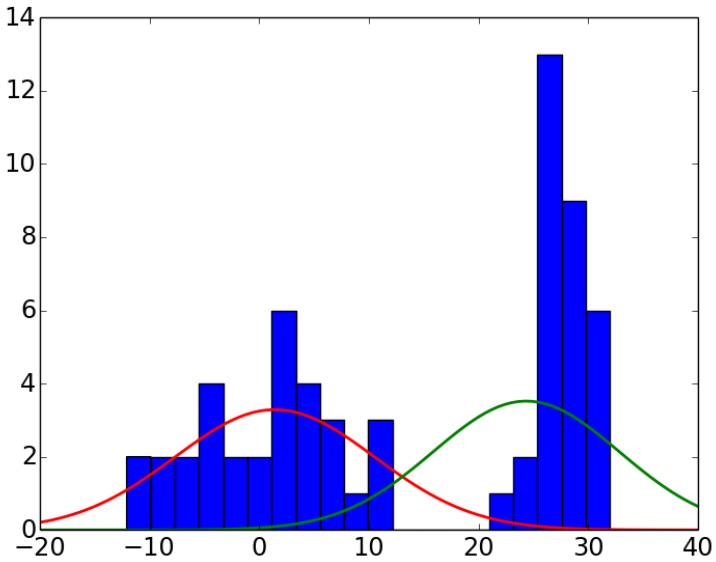
Temperature Example: Step 2



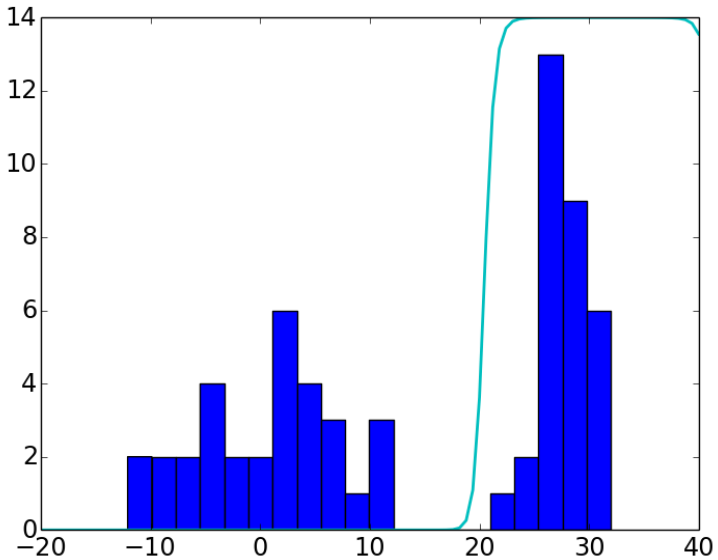


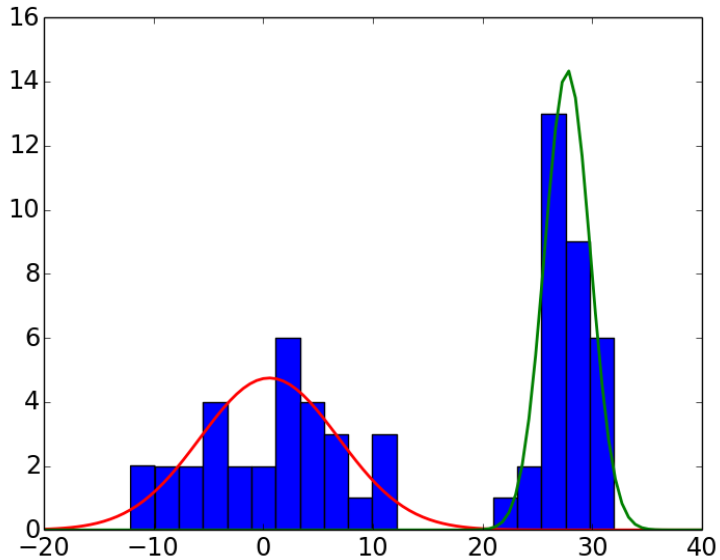
Temperature Example: Step 3



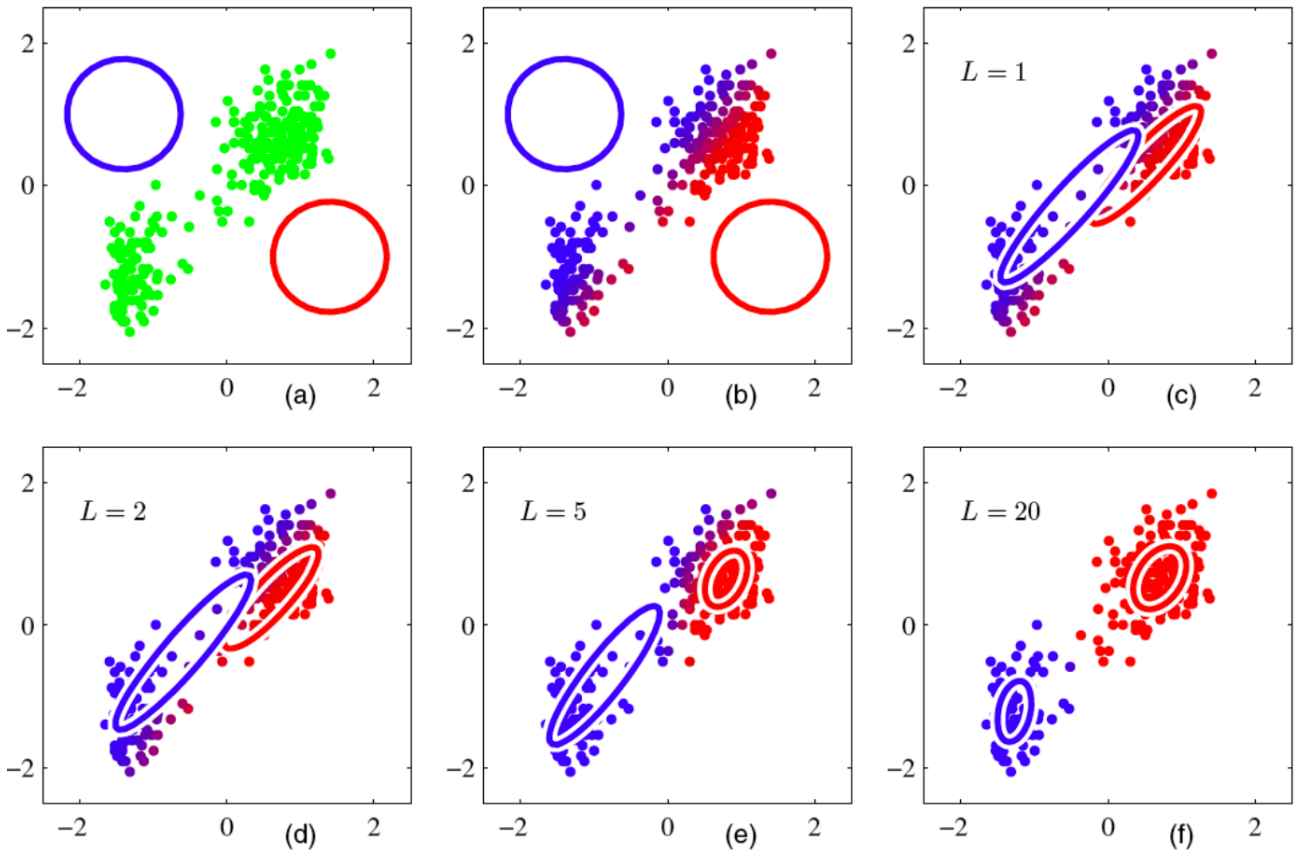


Temperature Example: Step 10





EM For Multivariate Gaussian



Relation to k-Means

The K-Means Algorithm:

- **Assignment step**: Assign each data point to the closest cluster
- **Refitting step**: Move each cluster center to the average of the data assigned to it

The EM Algorithm:

- **E-step**: Compute the posterior probability over z given our current model
- **M-step**: Maximize the probability that it would generate the data it is currently responsible for.

Relation to k-Means

The K-Means Algorithm:

- **Assignment step**: Assign each data point to the closest cluster
- **Refitting step**: Move each cluster center to the average of the data assigned to it

The EM Algorithm:

- **E-step**: Compute the posterior probability over z given our current model
- **M-step**: Maximize the probability that it would generate the data it is currently responsible for.

Can you find the similarities between the soft k-Means algorithm and EM algorithm with shared covariance $\frac{1}{\beta}\mathbf{I}$?

Both rely on alternating optimization methods and can suffer from bad local optima.

Section 4

Bernoulli Mixture Models (Lab 10)

Mixture Models

A GMM represents a distribution as a *mixture of Gaussians*

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$$

with π_k the **mixing coefficients**, where:

$$\sum_{k=1}^{K} \pi_k = 1 \quad \text{and} \quad \pi_k \geq 0 \quad \forall k$$

We can also have a mixture of **other distributions** instead of Gaussians, and fit the model via E-M!

- Expectation Step: Compute responsibilities
- Maximization Step: Estimate model parameters

Recall the Naive Bayes Model

Recall the Naive Bayes classifier:

$$p(\mathbf{x}, c) = p(c) \prod_j p(x_j|c)$$

Where $p(c)$ and each $p(x_j|c)$ is a Bernoulli distribution fit using labeled data.

Recall the Naive Bayes Model

Recall the Naive Bayes classifier:

$$p(\mathbf{x}, c) = p(c) \prod_j p(x_j|c)$$

Where $p(c)$ and each $p(x_j|c)$ is a Bernoulli distribution fit using labeled data.

If c is never observed, we call it a **latent variable**, or **hidden variable**, and again denote it with z instead.

Recall the Naive Bayes Model

Recall the Naive Bayes classifier:

$$p(\mathbf{x}, c) = p(c) \prod_j p(x_j|c)$$

Where $p(c)$ and each $p(x_j|c)$ is a Bernoulli distribution fit using labeled data.

If c is never observed, we call it a **latent variable**, or **hidden variable**, and again denote it with z instead.

Like before, by marginalizing out z, we get a density over the observables:

$$\begin{aligned} p(\mathbf{x}) &= \sum_k p(\mathbf{x}, z = k) = \sum_k p(\mathbf{x}|z = k)\, p(z = k) \\ &= \sum_k \prod_j p(x_j|z = k)\, p(z = k) \end{aligned}$$

Here, $p(z)$ is a categorical distribution parameterized by $p(z = k) = \pi_k$

Bernoulli Mixture Model (BMM)

A BMM is a finite mixture of random binary, independent features.

$$p(\mathbf{x}) = \sum_k \prod_j p(x_j|z=k)\, p(z=k)$$

Where $p(z=k) = \pi_k$, $\sum_{k=1}^{K} \pi_k = 1$ and $\pi_k \geq 0 \quad \forall k$

And each $p(x_j|z=k) = \theta_{jk}$ describes a Bernoulli distribution.

This defines a density over $\mathbf{x}$, so we can (again) fit the parameters using maximum likelihood. We try to match the data density of $\mathbf{x}$ as closely as possible.

These are the same parameters as in Naive Bayes!

BMM Likelihood

$$
\begin{aligned}
\ell(\{\pi_k, \theta_{jk}\}) &= \sum_k \sum_{i=1}^{N} \log \left\{ p(\mathbf{x}^{(i)} | z^{(i)}) p(z^{(i)} = k) \right\} \\
&= \sum_k \sum_{i=1}^{N} \log \left\{ p(z^{(i)} = k) \prod_{j=1}^{D} p(x_j^{(i)} | z^{(i)} = k) \right\} \\
&= \sum_k \sum_{i=1}^{N} \left[\log p(z^{(i)} = k) + \sum_{j=1}^{D} \log p(x_j^{(i)} | z^{(i)} = k) \right] \\
&= \sum_k \sum_{i=1}^{N} \log p(z^{(i)} = k) + \sum_k \sum_{j=1}^{D} \sum_{i=1}^{N} \log p(x_j^{(i)} | z^{(i)} = k)
\end{aligned}
$$

Fitting BMM parameters

Unlike in Naive Bayes, the classes z are *latent* and not observed. So, we will need to use the E-M algorithm to fit the parameters.

- **Expectation step (E-step)**: Compute the posterior probability over z given our current model - i.e. how much do we think each set of Bernoulli generates each data point. We use Bayes rule.
- **Maximization step (M-step)**: Assuming that the data really was generated this way, estimate the parameters π_k and θ_{jk}. We use maximum likelihood.

Expectation Step

Applying Bayes Rule, assign the **responsibility** $r_k^{(i)}$ of component k for data point i using the posterior probability:

$$r_k^{(i)} = p(z^{(i)} = k|\mathbf{x}^{(i)}) = \frac{p(z = k)\, p(\mathbf{x}|z = k)}{\sum_\ell p(z = \ell)\, p(\mathbf{x}|z = \ell)}$$

Maximization Step

Apply the maximum likelihood updates, where each component is fit with a weighted dataset. The weights are proportional to the responsibilities.

$$\begin{aligned} \pi_k &= \frac{1}{N} \sum_{i=1}^{N} r_k^{(i)} \\ \theta_{jk} &= ??? \end{aligned}$$

Complete this in the lab. This is very similar to parameter fitting in Naive Bayes!

Summary

Grouping data points into clusters, **with no observed labels**, is called **clustering**. It is an unsupervised learning technique.

Techniques for clustering:

- k-Means: For continuous features; if we expect the clusters to be spherical
- Mixture of Gaussians: allow more general shapes for clusters
- Other mixture models: allow various types of features and cluster distributions

Other commonly used methods: DBSCAN